

ABES ENGINEERING COLLEGE, GHAZIABAD

Department of Computer Science and Engineering

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

B.Tech 3rd Year Mini Project

HR Pulse – Enterprise Human Resource Management System

Project Title	HR Pulse – Enterprise Human Resource Management System
Student Name	Ayush Shukla
Roll Number	2400320100354
Branch / Department	Computer Science and Engineering (B.Tech, 3rd Year)
College Name	ABES Engineering College, Ghaziabad
Project Guide	MR. FAIZAN
Academic Year	2026–27

1. Abstract

HR Pulse is a web-based Human Resource Management System that digitizes the main HR activities of a small or medium organization. Manual methods such as paper registers and spreadsheets make attendance tracking, leave approval, payroll calculation, recruitment and appraisal slow and error-prone. The system offers a secure, role-based portal in which the HR Administrator can manage employees and departments, correct attendance, approve leave, generate monthly payroll with printable payslips, run a recruitment pipeline and record performance reviews. Employees use a self-service portal to punch in and out, apply for leave, view payslips and acknowledge reviews. The application is built with the MERN stack: React 18 with Vite and Tailwind CSS on the frontend, Node.js and Express.js REST APIs with JWT authentication on the backend, and MongoDB with Mongoose for storage. The expected outcome is a working, deployed application that reduces manual effort and calculation errors, keeps a clear approval trail and gives management dashboards and charts of workforce data.

2. Problem Statement

Many organizations still manage employee records, attendance, leave and payroll through paper or separate spreadsheets. This is time-consuming, data is difficult to search and update, leave approvals have no clear trail, and monthly salary calculation is repetitive and prone to mistakes. Employees cannot easily check their own attendance, leave balance or payslips.

The proposed system provides one digital platform where these processes are recorded, processed and reported, with different access for HR staff and employees.

3. Objectives

- Develop a user-friendly web application for core HR operations.
- Automate attendance, leave approval and payroll generation.
- Store employee and HR data efficiently in a central database.
- Reduce human errors and manual effort.
- Provide self-service access for employees.
- Generate dashboards, charts and exportable data for HR decisions.

4. Proposed Solution

How the system works. The React single-page application runs in the browser and communicates with an Express.js REST API under /api. A user logs in and the server returns a JWT token, which is sent with every later request. Middleware checks the token and the user's role before the request reaches the business logic. Data is stored in MongoDB through Mongoose models.

Main features. Dashboard with KPIs; employee directory with search, filters and CSV export; department management; punch-in/out attendance with late detection and admin correction; leave application and approval for Casual, Sick and Earned leave; salary breakdown with bulk payroll and printable payslip; job requisitions with a Kanban candidate pipeline; performance reviews with 1–5 star rating and goals; in-app notifications; analytics charts; dark and light themes.

Users. HR / Admin (full access) and Employee (personal portal).

Basic workflow. Admin adds employees and departments → employees punch in/out daily → employee applies for leave → admin approves or rejects and the employee is notified → admin runs bulk payroll for a month → employee views and prints the payslip → admin and employees review dashboards and reports.

Expected benefits. Faster processing, fewer errors, transparent approvals, one central record and instant insight into the workforce.

5. Scope of the Project

Implemented: authentication with two roles; employee and department management; attendance; leave management; payroll and payslip; recruitment pipeline; performance and appraisals; notifications; dashboard and reports. Data handled includes user accounts, employee profiles, attendance, leave, payroll, jobs, applications, reviews and notifications.

Outside the current scope: mobile app, e-mail/SMS alerts, biometric or geo-fenced attendance, statutory tax filing and integration with external payroll or banking systems.

6. Functional Requirements

ID	Functional Requirement
FR-01	The user shall be able to log in with email and password and log out; the system shall issue a JWT token on successful login.
FR-02	The system shall restrict features by role: HR/Admin gets full access and Employee gets only the personal portal.
FR-03	The Admin shall be able to add, view, update and delete employees, search and filter them, and export the list to CSV.
FR-04	The Admin shall be able to create departments with code, location and annual budget, and assign a department head.
FR-05	The Employee shall be able to punch in and punch out; the system shall calculate work hours and mark late arrival. The Admin shall be able to correct attendance manually.
FR-06	The Employee shall be able to apply for Casual, Sick or Earned leave and view the remaining balance; the Admin shall be able to approve or reject with remarks.
FR-07	The Admin shall be able to generate payroll in bulk for all active employees for a selected month; the Employee shall be able to view and print the payslip.
FR-08	The Admin shall be able to create job openings and move candidates through the stages Applied, Screening, Interview, Offered, Hired or Rejected.
FR-09	The Admin shall be able to create performance reviews with a 1–5 rating and goals; the Employee shall be able to add an acknowledgment comment.
FR-10	The system shall show in-app notifications with an unread count and dashboards and charts for attendance, department headcount/budget and payroll.

7. Non-Functional Requirements

Performance: Normal page loads and API requests should complete within a few seconds on a standard internet connection.

Security: Passwords are stored as Bcrypt hashes, protected routes require a valid JWT, and only authorized roles can reach restricted features.

Usability: The interface is simple and consistent, with search, filters, one-click actions and a dark/light theme switch.

Reliability: Data is validated and saved in MongoDB so records stay consistent; sample data is seeded automatically on first run.

Compatibility: The responsive web application works on current desktop and mobile browsers such as Chrome, Edge and Firefox.

8. Technology Stack

Component	Technology
Frontend	React 18, Vite, Tailwind CSS, React Router v6, Axios, Recharts, Lucide Icons
Backend	Node.js, Express.js, JWT, Bcrypt.js, Morgan
Database	MongoDB with Mongoose (local / Atlas, in-memory fallback)
Programming Language	JavaScript (ES6+), JSX
IDE	Visual Studio Code
Version Control / Hosting	Git, GitHub; Vercel (frontend); Render configuration file included

9. System Architecture

USER → FRONTEND → BACKEND → DATABASE

The system follows a three-tier client–server architecture. The browser runs the React application, the Express.js server exposes REST APIs and enforces authentication and roles, and MongoDB stores all data.

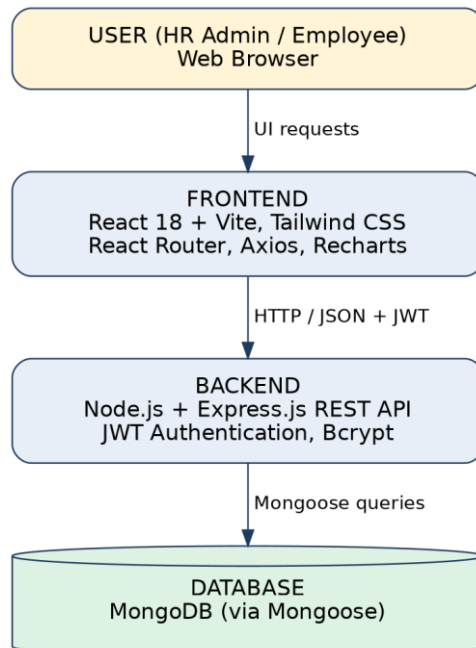


Figure 1: System architecture of HR Pulse

10. System Design

10.1 Use Case Diagram

Two actors use the system: the HR / Admin and the Employee. Both can log in and view a dashboard; the other use cases depend on the role.

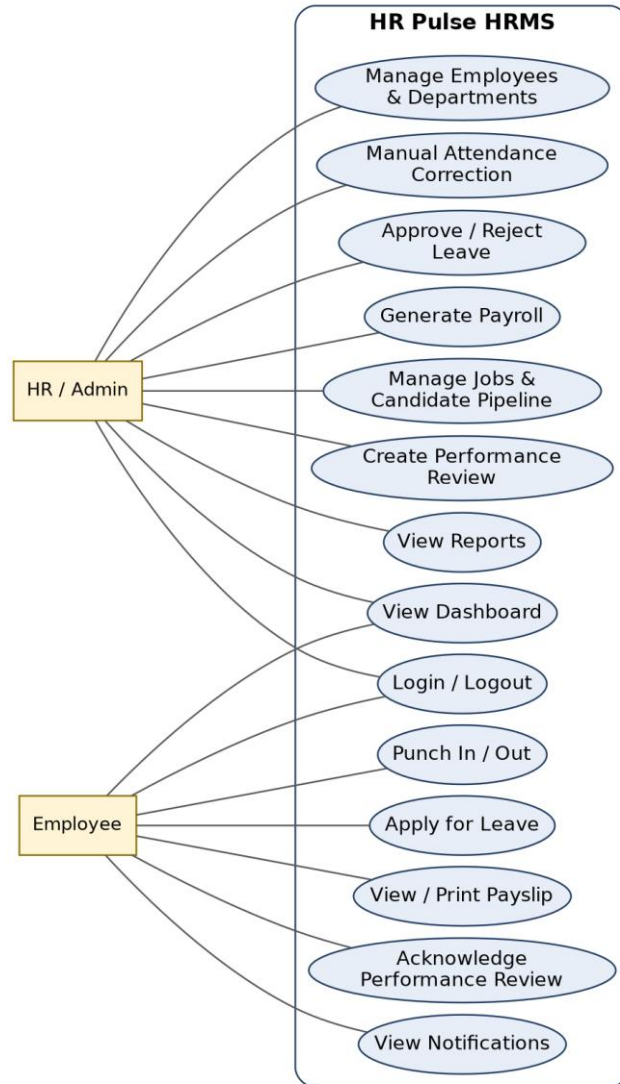


Figure 2: Use case diagram

10.2 Data Flow Diagram (Level 1)

The Level 1 DFD shows the seven main processes, the actors that feed them and the data stores they read or update.

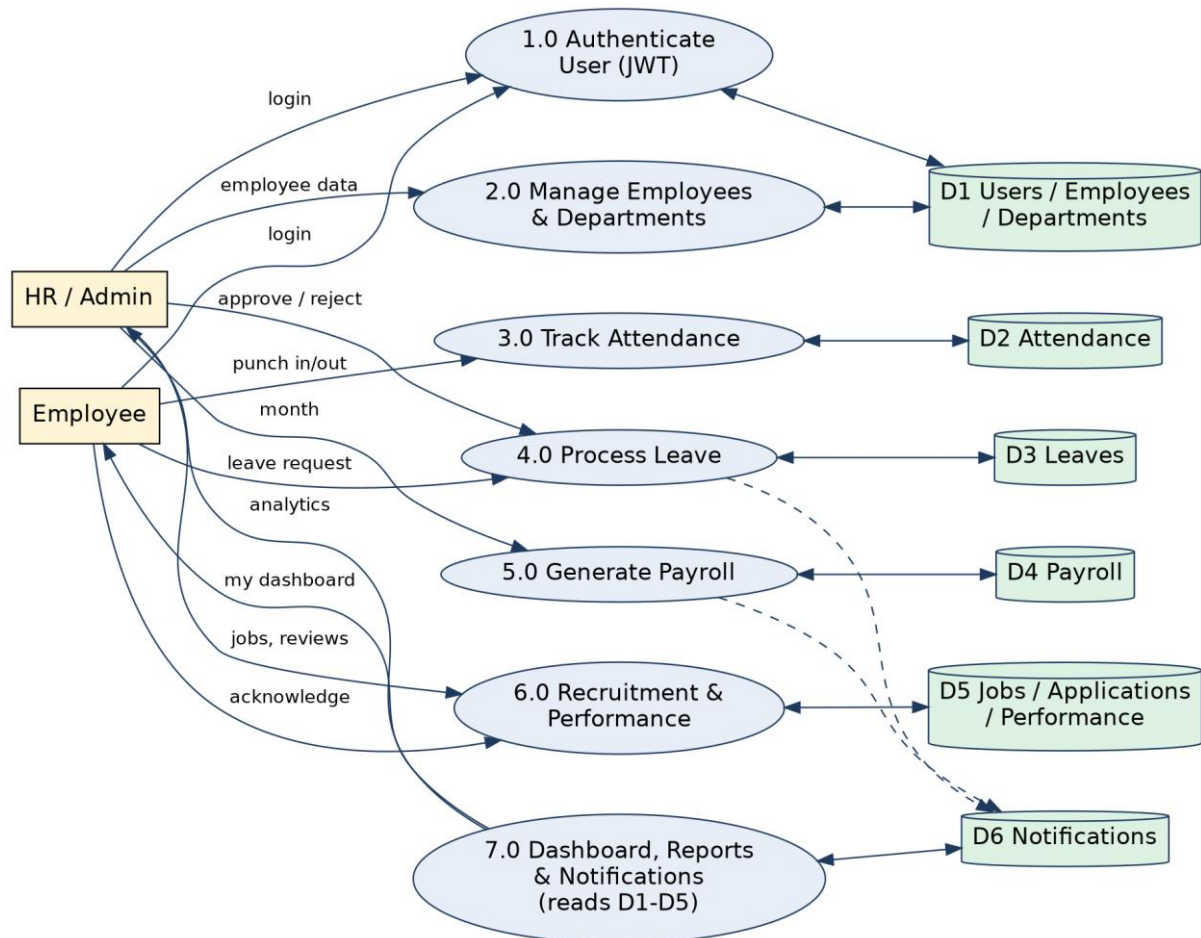


Figure 3: Level 1 data flow diagram

10.3 ER Diagram

The system uses MongoDB, so each entity below is a collection. Relationships are stored as references to the related document's `_id`.

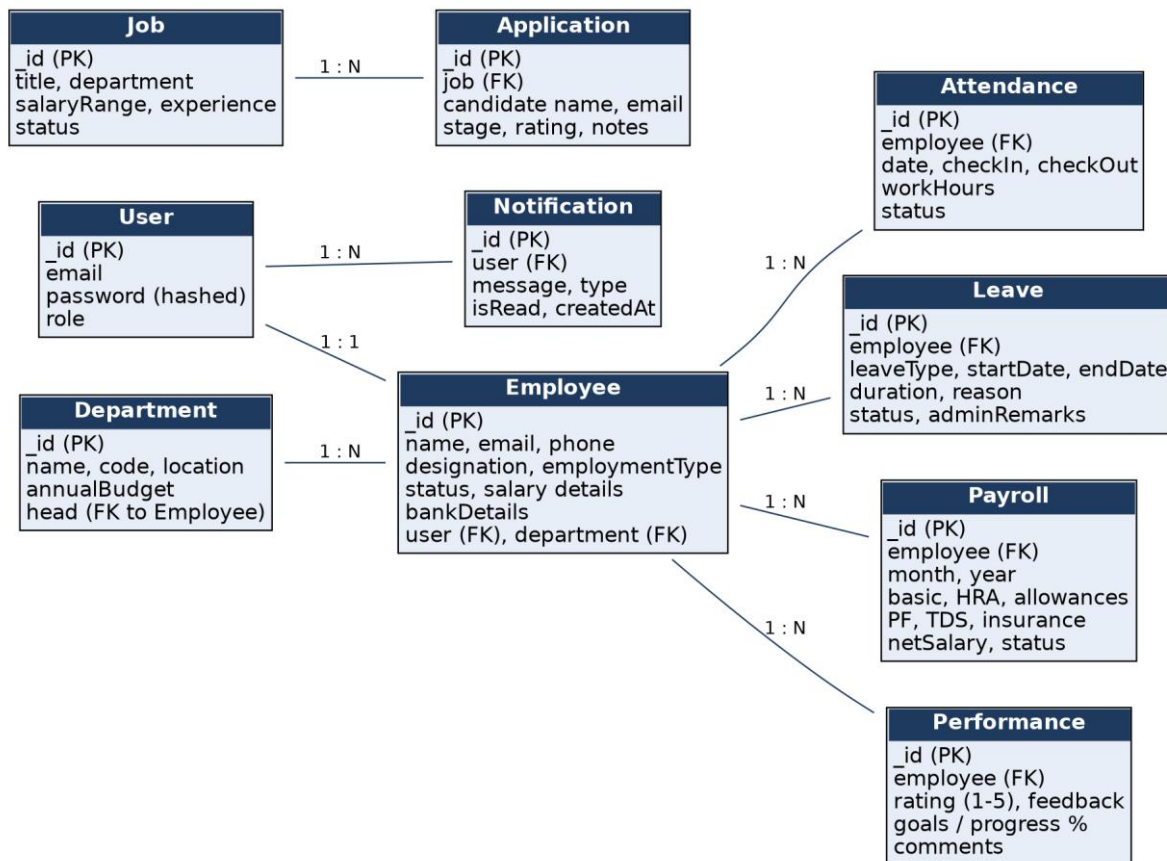


Figure 4: Entity–relationship diagram

10.4 Class Diagram

Not included. The project is a JavaScript MERN application whose data structures are shown in the ER diagram above, so a separate class diagram is not needed.

11. Module Description

Module 1 – Authentication

- User login and logout with JWT
- Register user account
- View/update profile and change password
- Role-based route protection

Module 2 – Employee & Department Management

- Add, update, delete and view employees
- Search and filter by name, email, department, type and status
- 360-degree employee profile; CSV export
- Create departments, assign head, view headcount and budget

Module 3 – Attendance & Leave

- Punch in / punch out with work-hours and late detection
- Attendance history with date and status filters; manual correction by Admin
- Apply for leave, view balances, approve or reject with remarks

Module 4 – Payroll

- Salary components: basic, HRA, conveyance, medical, bonus, PF, TDS, insurance
- Bulk payroll generation for a month
- Printable payslip

Module 5 – Recruitment & Performance

- Job requisitions; candidate pipeline with stage updates, rating and notes
- Performance reviews, goals with progress, employee acknowledgment

Module 6 – Dashboard, Reports & Notifications

- Admin and Employee dashboards
- Attendance, department and payroll charts
- In-app notifications with unread counter

12. Database Design

Important collections and their main fields are given below. Field names are described at the level of the application's features.

Collection	Key Fields	Description
User	_id, email, password, role	Login account with hashed password and role (Admin / Employee).
Employee	_id, name, email, designation, employment type, status, salary, bank details, user, department	Employee profile linked to a User and a Department.
Department	_id, name, code, location, annual budget, head	Organizational unit; head refers to an Employee.
Attendance	_id, employee, date, check-in, check-out, work hours, status	Daily record: Present, Late, Half Day, Leave or Absent.
Leave	_id, employee, type, start/end date, duration, reason, status, remarks	Leave request and its approval outcome.
Payroll	_id, employee, month, earnings, deductions, net salary	Monthly payslip data for an employee.
Job	_id, title, department, salary range, experience, status	Job requisition.
Application	_id, job, candidate details, stage, rating, notes	Candidate in the recruitment pipeline.

Collection	Key Fields	Description
Performance	_id, employee, rating, feedback, goals, comments	Appraisal review with 1–5 rating and employee comment.
Notification	_id, user, message, type, read flag, created time	In-app alert for a user.

13. Testing

Test ID	Test Case	Expected Result	Status
TC01	Valid login (admin@hrms.com / admin123)	Dashboard opens with admin menus	Pass / Fail
TC02	Login with wrong password	Error message is displayed; no access	Pass / Fail
TC03	Admin adds a new employee	Employee is saved and appears in the directory	Pass / Fail
TC04	Employee punches in and then out	Time stamps and work hours are stored; late status shown if applicable	Pass / Fail
TC05	Employee applies for leave	Request is saved as pending and balance is shown	Pass / Fail
TC06	Admin approves a leave request	Status changes and the employee gets a notification	Pass / Fail
TC07	Admin runs bulk payroll for a month	Payslips are created for active employees; payslip can be printed	Pass / Fail
TC08	Employee opens an admin-only page	Access is denied / redirected	Pass / Fail
TC09	Move a candidate to the next stage	Candidate stage is updated in the pipeline	Pass / Fail

Note: Run each case on the deployed application and mark the Status column as Pass or Fail before submission.

14. Limitations

- Limited development time, so advanced HR features are not included.
- Basic JWT authentication; no two-factor login or refresh tokens.
- Notifications are in-app only; no e-mail or SMS.
- Payroll uses stored salary components and does not implement statutory tax rules.
- No dedicated mobile application and limited sample test data.

15. Future Scope

- Mobile application for employees.
- Cloud scaling and automated backups.
- Advanced analytics and AI-based insights such as attrition prediction.

- E-mail / SMS notifications and automated reminders.
- Stronger security and integration with external payroll, banking and biometric APIs.

16. References

- MongoDB Documentation – <https://www.mongodb.com/docs/>
- Mongoose Documentation – <https://mongoosejs.com/docs/>
- Express.js Documentation – <https://expressjs.com/>
- React Documentation – <https://react.dev/>
- Node.js Documentation – <https://nodejs.org/en/docs>
- JSON Web Token introduction – <https://jwt.io/introduction>
- Tailwind CSS Documentation – <https://tailwindcss.com/docs>
- Project repository – <https://github.com/Ayushukla7/HRMS>
- Live application – <https://hrms-seven-delta.vercel.app/>